

Extensible and Robust Evaluation of Similarity Queries

Daniel Schmitt
University of Salzburg, Austria
danielulrich.schmitt@plus.ac.at

Thomas Hütter
Software Competence
Center Hagenberg, Austria
thomas.huetter@scch.at

Nikolaus Augsten
University of Salzburg, Austria
nikolaus.augsten@plus.ac.at

ABSTRACT

We study the similarity join problem from a systems perspective. A similarity join retrieves all similar record pairs from two collections based on a given distance function. Existing solutions are often optimized for a single distance function and domain. Such monolithic solutions are limited both in their extensibility to new distance functions and their robustness against changing data characteristics.

To address these challenges, we introduce FAST, a similarity join algorithm designed for extensible and robust query evaluation. It leverages a novel abstraction called reductions, which transform similarity join problems from complex domains into simpler ones. A reduction graph is constructed to systematically enumerate query plans. Since cost models for similarity queries are typically unavailable, FAST employs runtime partitioning and a sampling-based strategy to select a near-optimal query plan with performance guarantees. It can utilize prebuilt indexes or build them on-the-fly, incorporating caching techniques to accelerate index construction and probing. Extensive experiments across diverse datasets, domains, and distance functions show that FAST consistently performs close to the optimal plan. Finally, two case studies highlight its utility for prototyping and its strength as a baseline for future similarity join algorithms.

PVLDB Reference Format:

Daniel Schmitt, Thomas Hütter, and Nikolaus Augsten. Extensible and Robust Evaluation of Similarity Queries. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://frosch.cosy.sbg.ac.at/dschmitt/fast-similarity-evaluation>.

1 INTRODUCTION

Similarity joins use a predefined *distance function* to retrieve all pairs of similar records from two datasets. The range similarity join matches two records if their distance falls within a given *threshold*. The choice of a distance function depends on the *domain* of the records and the specific application. For example, similarity joins on sets use Jaccard or overlap similarity [44], while string joins rely on edit [30] or Jaro similarity [16]. For joining hierarchical data, like JSON or XML, measures like the JSON edit distance [28] or pq-grams [7] have been used.

In this paper, we address the integration of similarity joins into a system, focusing on two main challenges that must be addressed:

- *Extensibility*: Since similarity measures are highly application-dependent and numerous measures have been proposed [16, 46, 48], the system must be designed to integrate new distance functions with minimal overhead.
- *Robustness*: The runtime performance of different similarity join algorithms for a given task largely depends on factors like the distribution of the input data and the similarity threshold. No single algorithm performs well in all scenarios, so an efficient query plan must be created.

Current solutions do not sufficiently address these challenges. State-of-the-art similarity joins [2, 3, 12, 18, 25–27, 29, 32, 35, 40, 41, 44, 50, 52, 63, 64, 69, 71, 72, 74] are typically monolithic, standalone algorithms that are optimized for one or a few distance functions within a single domain. Adding a new and possibly application-specific distance function requires developing a new algorithm. This approach does not scale beyond a few distance functions and domains. Designing a unified similarity join system that efficiently handles diverse data types remains a fundamental challenge.

Most similarity join techniques do not adapt to data properties; the user must choose an appropriate algorithm for a given task. A poor choice can lead to significant performance degradation. For example, set similarity join algorithms are sensitive to the number of different set elements, their distribution, the set sizes, and the user-defined threshold. Predicting the performance of an algorithm on a given dataset is difficult, and choosing the wrong technique can lead to slowdowns of multiple orders of magnitude [52].

A common approach for query planning in database systems enumerates equivalent plans and uses a cost model to select the best plan. Unfortunately, cost models cannot be easily applied to similarity queries for two reasons: (1) No cost models that compare different similarity joins are available [5]. Note that estimating the join cardinality [31, 45, 56, 67] is not enough as the output size is the same for all physical similarity join operators. (2) The join cost depends on the distance function. Extending the system to include new application-specific distances requires the development and integration of new cost models, which is expensive.

In this paper we propose FAST, a novel approach for integrating similarity joins into a system that ensures flexibility and performance. Our solution relies on modularization and the novel concept of *reductions* to achieve extensibility. A sample-based approach to query planning avoids hard-to-obtain cost models and adapts gracefully to new distance functions and operator implementations. The query planner of FAST is not limited to physical operator selection, but can combine techniques from different similarity join algorithms into new and more efficient query plans.

Modularization and Reductions. We observe that most algorithms for similarity joins follow a similar pattern: A single- or multi-level inverted list index is used to perform an indexed nested-loop join

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

in a filter-verify framework. A common pattern in the filtering step transforms the join problem to a simpler domain (e.g., trees are transformed to strings or sets) such that the join result in the simpler domain (called the candidate set) is a superset of the solution in the original domain; a verification step removes false positives.

Previous similarity join algorithms hardcode these steps in a monolithic solution. We follow a different approach and introduce reductions as an explicit step into our modular framework. A *reduction graph* relates different domains and distance functions, and a path in the graph that ends in a leaf is an evaluation plan. Figure 1 shows the reduction graph of our prototype implementation: the four leaves are physical operators, and the edges are reductions (possibly within the same domain). Figure 2 shows the nine different evaluation plans available for a tree edit distance (TED) join in our prototype.

The reduction graph approach has several advantages over hard-coded reductions in monolithic solutions: (1) *Plan enumeration*: Instead of limiting the execution to a single reduction, all known reductions are leveraged to enumerate candidate plans and select the best plan. (2) *Extensibility*: Integrating a new distance function only requires finding a reduction to a known distance function in the graph. As an illustration, we extend our prototype with the widely used Jaro similarity for strings [16]. (3) *Code redundancy* is avoided. For example, a string join based on q -grams does not need to deal with identifying pairs of similar q -gram sets: defining a new reduction is enough to leverage all available set similarity join operators.

Query Planning. To address the issue of unavailable cost models, we propose a sampling-based strategy: A plan is scheduled for a short timeslice and its progress is recorded. Query plans are modeled as a Markov decision process and are selected based on their historic performance. Using the *Upper Confidence Tree* (UCT) [39] algorithm guarantees eventual convergence to the best query plan.

We consider two scenarios: All indexes are prebuilt or partial indexes are constructed on-the-fly when required. Indexing on-the-fly is challenging: (1) As indexing does not constitute progress in the evaluation of the query (measured as the number of join result tuples), assessing the performance of an algorithm is harder, and (2) the overhead related to index probing is incurred multiple times for each timeslice. We propose a technique to build and probe two indexes incrementally to support general (non-self) joins. To speed up the computation and reduce the overhead, we propose caching strategies for (partial) indices, reusable reduction data of similar query plans, and so-called *signatures* for index probing.

We argue that FAST should serve as a baseline for new similarity join algorithms. Existing implementations assume a single query plan; FAST considers them all, better reflecting previous work. Extensions of FAST focus on the novel part of an algorithm and share existing modules, leading to comparable results. We support our claim with two case studies: An extension of FAST to the Jaro similarity between strings outperforms the state-of-the-art algorithm by orders of magnitude. We further compare FAST to three recent similarity join algorithms and show that two of them do not provide significant speedup w.r.t. an efficient query plan over well-known techniques.

In summary, our contributions are the following:

- Our FAST (Fully Automatic Similarity Transformation) algorithm leverages the novel concept of a *reduction graph* to enumerate all possible evaluation plans.

- For *query planning*, we propose a feedback strategy based on runtime partitioning and provide performance guarantees w.r.t. the optimal plan. We do not require cost models.
- To reduce the overhead of partial indexes, we propose a *caching strategy* that stores intermediate results across repeated selections of the same evaluation plan.
- Extensive experiments on twenty datasets, three domains and different distances show the robustness of our approach: On average, FAST is only 12% slower than the optimal plan.
- Two case studies show the *extensibility* of FAST and its value as a *benchmark*: Only one of the four tested state-of-the-art algorithms outperforms the query plans proposed by FAST.

The remaining paper is organized as follows. We provide background information and define our objectives in Section 2. The *reduce-filter-verify* framework is defined in Section 3. Section 4 defines reduction graphs for the enumeration of query plans that are evaluated on prebuilt indexes in Section 5 and on dynamic indexes in Section 6. Sections 7-9 present experimental results and show FAST’s capability as a baseline and for prototyping. Related work is presented in Section 10, Section 11 concludes the paper.

2 BACKGROUND AND PROBLEM STATEMENT

Similarity Search. Similarity search deals with the problem of matching pairs of similar records (r, s) from some datasets R, S of some domain $\mathcal{T}$. The similarity of two records is assessed by a distance function $d: \mathcal{T}^2 \rightarrow \mathbb{R}_0^+$. $\mathcal{D}$ denotes the set of all distance functions. We focus on *range similarity join* queries: Given datasets $R, S \subseteq \mathcal{T}$ and a distance threshold ϵ , the range similarity join $R \bowtie_{\epsilon}^d S$ finds all pairs $(r, s) \in R \times S$ with $d(r, s) \leq \epsilon$. In this paper, we assume self-joins ($R = S$) that are common for applications such as entity resolution [71]. However, all results generalize to non-self joins.

Domains and Distance Functions. We focus on sets, strings, and trees as they are widely used in similarity search [6]. In particular, these domains form a ladder of increasing structure: Every set can be represented as a string and every string can be represented as a tree without losing information. Many algorithms for similarity search take the opposite direction: A reduction in structure allows for faster computations, e.g., by reducing the tree edit distance with cubic runtime to the quadratic time string edit distance [40].

Commonly studied distance functions on strings and trees are their respective edit distances. For sets, we consider all *symmetric* [15] similarity measures. Symmetric similarity measures on sets can be rewritten as a minimum required overlap given the sizes of two sets. Symmetric similarity measures include common similarity measures like Jaccard, Cosine, and Dice, or the Hamming distance.

Problem Statement. The goal of this work is to develop a system that is capable of computing similarity joins on sets, strings, and trees with edit distance and symmetric set similarities. Furthermore, the system should satisfy the following properties: (1) **Extensibility**: Support for new distance functions can be added without significant rework. (2) **Robustness**: The system adapts to a given query, e.g., its characteristics, and computes its result efficiently.

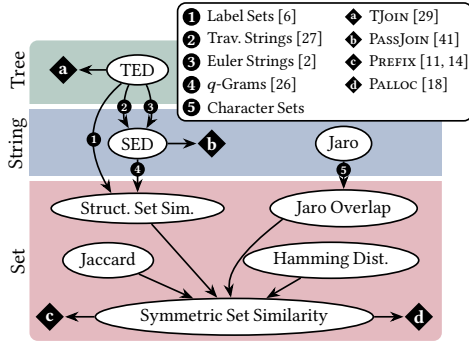


Figure 1: Reduction Graph for Sets, Strings, and Trees.

3 REDUCE-FILTER-VERIFY FRAMEWORK

Most algorithms for similarity search are based on the *filter-verify* [29, 41, 43, 51] framework. We extend this framework by an explicit *reduce* step, which is an implicit part in the filter step of previous algorithms. A *reduce-filter-verify* algorithm consists of three steps:

Reduce: The dataset, distance function, and threshold are reduced by a *reduction*. A reduction from domain $\mathcal{T}_1$ to $\mathcal{T}_2$ is a (partial) function $\rho: \mathcal{T}_1 \times \mathcal{D} \times \mathbb{R}_0^+ \rightarrow \mathcal{T}_2 \times \mathcal{D} \times \mathbb{R}_0^+$. It maps a record from $\mathcal{T}_1$, a distance function on $\mathcal{T}_1$, and a threshold onto a record from $\mathcal{T}_2$, a distance function on $\mathcal{T}_2$, and a new threshold. Typically, domain $\mathcal{T}_2$ has less structure than $\mathcal{T}_1$; examples include converting trees to sets [8, 29], strings [32, 40, 72], or vectors [40]. We simply write $\rho(r)$, $\rho(d)$, and $\rho(\epsilon)$ if the other arguments are clear from context; $\rho(R)$ denotes the dataset computed by pointwise application of ρ to the elements in the dataset R . A reduction has the property of *similarity supersets*:

$$R \succsim_d^\epsilon S \subseteq \rho(R) \succsim_{\rho(d)}^{\rho(\epsilon)} \rho(S)$$

Filter: The filtering step computes the similarity join after reduction: $\rho(R) \succsim_{\rho(d)}^{\rho(\epsilon)} \rho(S)$. By the similarity supersets property, all pruned pairs are true negatives, i.e., they are not part of $R \succsim_d^\epsilon S$. We assume that this step is index-based: Given a query $\rho(q)$, the index returns a set of candidates including all truly similar pairs. We will state further assumptions on the index structure in Sections 5 and 6. All current state-of-the-art algorithms for similarity search on discrete domains satisfy those assumptions [12, 19, 26, 29, 35, 41, 50, 52, 74].

Verify: The candidates produced by the filtering step include all true positives, but also false positives. The false positives are removed by computing the similarity on the original domain and comparing against the threshold. As computing the similarity on the reduced data is typically faster than on the original data, candidates are often verified on the reduced data first.

4 QUERY PLAN ENUMERATION

Any algorithm following the reduce-filter-verify framework can be broken up into three parts: reduce, filter, and verify. Hence, any such algorithm can be seen as a query plan composing a specific reduction step, a filtering algorithm, and a verification procedure. We introduce the *reduction graph* to enumerate all valid query plans for a given query. Informally, a reduction graph connects distance functions using reductions. Distance functions connected to some algorithm can be evaluated by said algorithm without further reduction.

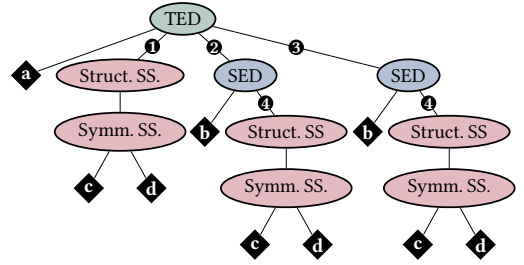


Figure 2: Query Plans/MDP for TED represented as a Tree.

Definition 4.1 (Reduction Graph). A reduction graph is a directed acyclic edge-labeled multigraph. It distinguishes two types of nodes: (1) A distance node corresponds to a distance function on some domain. (2) Algorithm nodes correspond to a concrete join algorithm. A distance node connects to an algorithm node if a similarity join with that distance function can be evaluated by the algorithm. Edges between distance nodes correspond to reductions between the distance functions and their respective domains.

Paths in the reduction graph starting at some distance node for some distance function d and ending in an algorithm a correspond to query plans of a reduce-filter-verify algorithm: The reduction step is the composition of all reduction edges on the path. The filter step is implemented by algorithm a . For the verification step, we reverse the path and evaluate the reduced distance function for each distance node, pruning dissimilar pairs in the process. To enumerate all possible query plans $\mathcal{P}$ for a given distance function, we perform standard depth-first search from the corresponding distance node. The correctness of such query plans follows from compositionality of reduction functions and the correctness of filtering algorithms.

Example 4.2. Consider the reduction graph depicted in Figure 1. There are nine possible query plans for a similarity query using the *tree edit distance* (TED). Figure 2 visualizes all plans as a tree.

- **Trees:** Use TJOIN [29] without any reduction.
- **Sets:** Reduce the trees to sets of node labels and use either PREFIX [11, 14] or PALLOC [18].
- **Strings:** Use traversal strings [27] or Euler strings [2] to convert the trees to strings. Then, use the *string edit distance* (SED) algorithm PASSJOIN [41]. Alternatively, perform another reduction to sets using *q*-grams [26]. In the latter case, filtering is performed using PREFIX [11, 14] or PALLOC [18].

5 STATIC INDICES QUERY EVALUATION

Given a query, the reduction graph enumerates all query plans $\mathcal{P}$ to evaluate the query. Similarity join algorithms generally do not have cost models that could be leveraged to select the best plan. Instead, we propose to use a sampling-based approach that executes one query plan for a short period of time, measures its performance, and bases its future selections on the previous performance of a plan. The selections based on previous performance uses the Upper Confidence Tree (UCT) [39] algorithm to ensure eventual convergence to the fastest plan. In this section, we provide background information on UCT, define assumptions on the similarity join algorithm implementing the filtering step, and show how near-optimal query plans can be selected.

5.1 Background on UCT

UCT [39] is the backbone of our plan selection strategy. UCT finds near-optimal solutions for a (finite horizon) *Markov Decision Process* (MDP). For our purposes, an MDP is a tuple $(\mathcal{S}, \mathcal{A}, \delta, \mathcal{R})$ consisting of a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a deterministic transition function $\delta: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$, and a stochastic reward function $\mathcal{R}$ assigning probability distributions for rewards in $[0, 1]$ to states.¹

The representation of the set of query plans $\mathcal{P}$ as a tree is closely related to its MDP (cf. Figure 2): In the tree, nodes are distance functions or algorithms, edges to inner nodes are reductions, and root-leaf paths are query plans. In the MDP, nodes correspond to prefixes of such paths, actions are either reductions or algorithms, and transitions are the edges connecting the path prefixes. Rewards for all inner nodes are zero. The rewards on the leaf nodes represent the performance of the leaf’s query plan. We will define how rewards are calculated in Section 5.3.

UCT [39] solves MDPs by balancing the aspects of exploration – exploring parts of the tree with little information to aid in future decisions – and exploitation – repeating choices that led to large rewards in the past. To this end, UCT maintains both the average reward r and the number of visits v for each node. To select a path, UCT starts in the root of the UCT tree and always selects child c with parent p maximizing the average reward r_c plus a bias term $w\sqrt{\log(v_p)/v_c}$ with weight w . The bias is derived from a Chernoff-Hoeffding bound and represents the certainty of the current average [17]. With increasing visits to c , the certainty increases and the bias decreases. Theoretical guarantees require setting w proportional to the height of the tree [39], but empirically, application specific tuning of w can lead to improved performance [20]. After achieving a reward at a leaf node, the statistics of all nodes along the path are updated.

The performance of UCT and similar algorithms is measured in the *regret* of their decisions. Regret of a learning process is the difference in rewards between the selected policy of the learning algorithm and the optimal policy. We focus on *cumulative regret*, which measures the difference in expected maximal rewards compared to the achieved rewards *over all choices* of the algorithm. For UCT, the cumulative regret after n path selections is $O(\log n)$ (Theorem 6 in [39]). While other algorithms converge faster than UCT [23, 24], they optimize for a different notion of regret that is not suitable for our purpose.

5.2 Assumptions on Similarity Join Algorithms

As stated in Section 2, we assume that all algorithms implementing the *filter* step in the *reduce-filter-verify* framework are index based. It should satisfy the following requirements: (1) **Independent-Index**: A dataset can be indexed in advance without knowledge about the future queries. (2) **Total-Recall**: For a given query record, the algorithm returns a superset of all similar records in the indexed dataset, possibly including false positives. (3) **Prefix-Search**: The index allows queries on a *prefix* of the dataset. In other words, for a given dataset $R = \{r_1, \dots, r_n\}$, the algorithm can perform queries on any $R' = \{r_1, \dots, r_i\}$ with $i \leq n$. Inverted list indexes support such queries by storing the records sorted by record ID.

¹We use the same formalism of MDPs as Trummer *et al.* [61] following Silver [55]. In more general scenarios, the transition function might be stochastic and rewards of moving to some state might depend on the previous state and the action taken.

5.3 Finding Efficient Query Plans

We first discuss a variant of FAST that assumes that the indexes required for all query plans $p \in \mathcal{P}$ are available. Conceptually, computing a (similarity) join requires comparing all pairs of records. Figure 3a visualizes all required comparisons for a similarity self-join. Due to the symmetry of distance functions, only the upper triangular matrix needs to be computed. The explicit computation of all pairs of records, index-based similarity algorithms use an index that only returns promising pairs. Still, the area of the upper triangular matrix represents the full work required for the join.

Next, we address the following problem of finding a good evaluation strategy: Given a set of query plans $\mathcal{P}$, e.g., computed by enumerating all paths starting from the query’s distance node in the reduction graph (cf. Section 4), we want to evaluate the similarity join using some query plans $p \in \mathcal{P}$ that – in hindsight – approximately perform as well as the single optimal plan $p^* \in \mathcal{P}$. To this end, we use similar ideas as Trummer *et al.* [61] by evaluating partial joins with timeslices using different query plans. To compute a join, we first construct an UCT instance corresponding to the MDP of $\mathcal{P}$ (cf. Section 5.1). We then repeatedly select a plan $p \in \mathcal{P}$ using the UCT instance. A plan consists of (1) the reduction ρ implemented as the composition of any number of reduction functions, (2) an algorithm implementing the filter step, and (3) a verification procedure. After selecting the current plan, the plan is executed by following the *reduce-filter-verify* framework for the next non-processed record r_i (cf. Figure 3a). We reduce r_i to $\rho(r_i)$ probe $\rho(r_i)$ against the index on $\rho(R)$, obtaining a set of candidates. To avoid symmetric result pairs, we only probe $\rho(r_i)$ against the prefix of $\rho(R)$ consisting of the first $i-1$ records. According to the assumptions in Section 5.2, prefix index queries can be performed efficiently. To verify the results, we proceed as described in Section 4: Let $\rho = \rho_1 \circ \dots \circ \rho_m$ and $\rho^k = \rho_1 \circ \dots \circ \rho_k$ for $k \leq m$. We verify all candidates starting from the lowest level with reduction ρ^m and continue with the higher levels $\rho^{m-1}, \rho^{m-2}, \dots$ until reaching the identity reduction ρ_0 . The verification at level k removes all candidates s with $\rho^k(d)(\rho^k(r), \rho^k(s)) > \rho^k(\epsilon)$.

When probing from left to right (cf. Figure 3a), we probe against increasingly larger indexes that can yield larger candidate sets. To reduce this skew of increasing verification effort, we alternately pick a record from the left and the right side of the unprocessed records. We continue probing until we reach a timeout.

After the timeout, we evaluate the plan’s performance and update the UCT instance. The chosen plan’s performance for a timeslice is the number of conceptually compared pairs of records (cf. Figure 3a, hatched area). As we alternate between both sides of R , we have $m(|R|-1)$ non-reflexive and non-symmetric pairs after processing $2m$ records with m records from each side. As UCT expects rewards between 0 and 1, we normalize by the size of the upper triangular matrix $|R|(|R|-1)/2$. This results in the reward function $\frac{2m}{|R|}$.

Discussion. The assumption of prebuilding all possible indexes, while impractical for large sets of query plans, allows for a simple analysis. In particular, the query plans selected by UCT perform asymptotically as well as the optimal query plan. The key requirement for this result is that the work of all plans is shared: The progress of each plan towards computing the join result can be reused efficiently. Once a record has been probed by any query plan, it will never need to be probed again. This holds in case of static indexes.

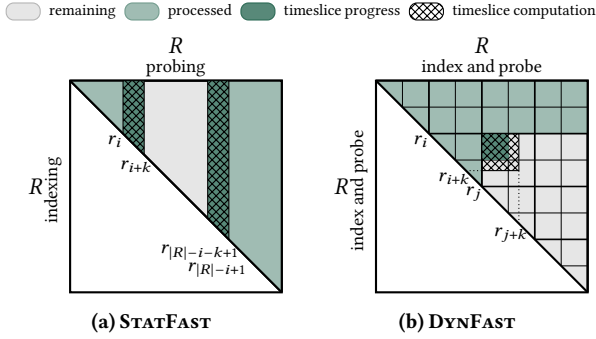


Figure 3: Processing Pattern of STATFAST and DYNFAST

For dynamic indexes as described in Section 6, some parts of the result must be discarded and recomputed later.

5.4 Regret Analysis

We analyze the expected regret of FAST with prebuilt indexes (called STATFAST). Our goal is to perform approximately as well as the best query plan p^* in $\mathcal{P}$. In other words, we want to compare the policy of always selecting p^* on every timeslice to the plans selected by STATFAST. This notion is exactly captured by *cumulative regret*, i.e., the difference in the sum over all rewards achieved during execution. As the reward for each timeslice is defined to be the size of the processed partial join, rewards measure the throughput of a policy. We will then relate throughput and execution time.

Regret Model and Assumptions. We denote optimal quantities with $*$ -superscripts, e.g., our approach requires n timeslices while the optimal runtime is n^* . We analyze regret under the assumption that runtime is high compared to the number of plans. In particular, we assume that we achieve the asymptotic regret bounds of UCT and do not suffer from transitory regret [17]. Furthermore, we assume that the runtime related to traversing and updating the UCT tree and switching between plans is negligible. Finally, we assume that the average reward of the optimal algorithm cannot be arbitrarily small ($\Omega(1)$). To this end, timeslices are increased for large input sizes. This is in line with related work utilizing UCT in database systems [61].

LEMMA 5.1. *STATFAST has runtime regret of $O(\log n)$.*

PROOF. The proof follows the structure of Trummer *et al.* [61]. The difference of the runtime n achieved by FAST and the optimal runtime n^* is the regret $n - n^*$. Let $\mathcal{R} = \sum_{p \in \mathcal{P}} \mathcal{R}_p$ be the total reward FAST achieved over all plans $p \in \mathcal{P}$. $r = \frac{\mathcal{R}}{n}$ is the average reward achieved by FAST in a timeslice. Similarly, we have the optimal quantities $r^* = \frac{\mathcal{R}^*}{n^*}$. For STATFAST, rewards directly correspond to relative size of the computed partial join result. Once a total reward of one is achieved, execution is terminated as the full join has been computed. Hence, $\mathcal{R} = \mathcal{R}^* = 1$, $r = \frac{1}{n}$, and $r^* = \frac{1}{n^*}$. By our assumption of achieving the asymptotic regret bounds of UCT, we have $r^* - r = O(\frac{\log n}{n})$ [39]. Hence, we have $r = r^* - (r^* - r) = r^* - O(\frac{\log n}{n})$. We now bound the regret $n - n^*$. We have $n - n^* = n(1 - \frac{n^*}{n}) = n(1 - \frac{r^*}{r}) = n(1 - \frac{r^* - O(\frac{\log n}{n})}{r^*}) =$

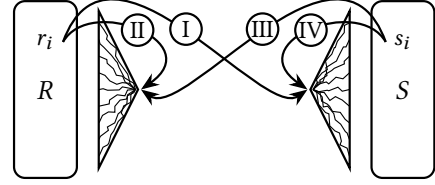


Figure 4: Two-Sided Incremental Probe-and-Insert

$n(1 - 1 + \frac{O(\frac{\log n}{n})}{r^*})$. By our assumption of $r^* = \Omega(1)$, this is bounded by $O(\log n)$. $\square$

We also express this additive regret in the more intuitive form of a multiplicative regret, bounding the asymptotic runtime ratio of FAST and the fastest algorithm.

LEMMA 5.2. *STATFAST has bounded expected multiplicative runtime regret n/n^* that converges to 1 with $n \rightarrow \infty$.*

PROOF. We have $n - n^* \leq c \log n$ for some constant c . Hence, $\frac{n}{n^*} \leq \frac{1}{1 - \frac{c \log n}{n}}$. With $n \rightarrow \infty$, the right term converges to 1. $\square$

6 DYNAMIC INDICES QUERY EVALUATION

In this section, we remove the assumption that the indexes of all query plans are built in advance. Our approach follows the typical procedure in similarity joins of alternately building and probing the index. After clarifying the assumptions, we introduce a dynamic version of FAST that builds indexes on-the-fly, analyze its regret, and reduce its overhead for practical applications.

6.1 Assumptions on Similarity Join Algorithms

In addition to the assumptions defined in Section 5.2, we require: (4) **Updateability**: Efficient incremental updates are supported: We can alternate between indexing new records and probing records without knowing all indexed records in advance. (5) **Range-Search**: In addition to index prefixes (c.f. Assumption (3)), any contiguous range of record IDs can be probed efficiently. If an algorithm is used to index a dataset $R = \{r_1, \dots, r_l\}$, it supports efficient queries on any contiguous subset $R' = \{r_j, \dots, r_k\}$ with $i \leq j \leq k \leq l$. Inverted list indexes support such queries by storing the lists sorted by record ID, finding the first record in the range using binary search, and skipping the remainder of the list after finding an ID higher than k .

6.2 Plan Evaluation with Index Construction

Missing prebuilt index structures poses an issue for evaluating the performance inside a timeslice. Index construction in itself does not constitute progress of the similarity join and thus should not be part of the reward. Otherwise, the index-construction part of the reward would be plan specific and would thus lead to regret bounds dependent on the number of query plans. Instead, we interleave building and probing the index to get a sample of the average performance of a specific query plan.

Incremental Probe-and-Index. When probing a record, the current state of the index will not encompass the full relation in general. Hence, we cannot simply partition the space of all record pairs like

Algorithm 1: Block Scheduling

Data: Stack $\mathcal{S}$, initially $((((1,1),(|R|,|R|)),\{UL,UR,LR\}))$

```
1 Function FullyProcessed():  
2   do pop from  $\mathcal{S}$ ; remove popped subblock from  $\mathcal{S}^\top$   
3   while  $\mathcal{S}^\top$  fully processed  
4 Function PartiallyProcessed(computed block  $B_c$ ):  
5   while  $\mathcal{S}^\top$  not covered by  $B_c$  do  
6     // blocks are processed from UL to LR  
7     push UL subblock of  $\mathcal{S}^\top$  onto  $\mathcal{S}$   
8     FullyProcessed() //  $\mathcal{S}^\top$  was fully processed  
9 Function NextBlock():  
10  if  $\mathcal{S}^\top$  partially processed then push its next subblock onto  $\mathcal{S}$   
11  return  $\mathcal{S}^\top$ 
```

for the non-incremental setting in Figure 3a. Instead, we will alternate between building, probing, and verifying the next record. A simple implementation of this approach has been used in various similarity join algorithms [11, 18, 41, 71], but it can only be used to compute self-joins. We will refer to this approach as Inc1S (*incremental one-sided*) and introduce it as a special case of Inc2S (*incremental two-sided*) that we will discuss next. In our setting, Inc1S can only compute triangles along the diagonal in Figure 3b. For all other cases, we have to compute a non-self join between different parts of the relation using the following variation Inc2S that was already briefly discussed by Bouros *et al.* [13] in the context of the prefix filter. Figure 4 shows the overall structure. Given two relations R, S of equal size, we ① take the i -th record r_i from R , probe it against the current index on S , and verify the candidates. In step ②, we insert r_i into the index on R . Step ③ and ④ correspond to ① and ② with reversed roles of R and S . We proceed to the next record of both sides until both relations are processed. Inc1S works like Inc2S by setting $R=S$ and skipping steps ③ and ④. Both Inc1S and Inc2S require the assumed updateability of the index structure.

To conceptually compare all pairs of points in the upper triangular matrix in Figure 3b, we use both Inc1S and Inc2S. Given some pair of starting records (r_i, r_j) , we either execute Inc1S if $i=j$ and Inc2S otherwise until a timeout is reached. After the timeout, we have reached some record (r_{i+k}, r_{j+k}) s.t. all pairs in the square with the corners (r_i, r_j) and (r_{i+k}, r_{j+k}) have been evaluated (cf. Figure 3b). For self-joins using Inc1S, a triangle is computed. From now on, we will refer to both squares and triangles as *blocks*. Computing the full join requires *covering* the full upper triangular matrix with blocks. The sizes of the blocks will differ significantly both between different query plans and different parts of the matrix.

Block Scheduling. To avoid fragmentation and bookkeeping and guarantee covering all required pairs of records, we propose the following partitioning scheme of the matrix depicted in Algorithm 1. Algorithm 1 only encapsulates the scheduling of blocks; its functions are called as part of FAST. We maintain a stack $\mathcal{S}$ of blocks that still have to be filled. Each block consists of four subblocks UL, UR, LL, LR of equal size constructed by halving both sides of the block. In addition to the block, $\mathcal{S}$ also stores the set of subblocks of each block that have not been computed yet.

Initially, $\mathcal{S}$ contains the full dataset $((1,1),(|R|,|R|))$ with all subblocks $\{UL, UR, LR\}$; LL is missing to avoid symmetric pairs of self-joins. To schedule the next block (cf. Algorithm 1, NextBlock), we peek at the top $\mathcal{S}^\top$ of the stack. If $\mathcal{S}^\top$ has no computed subblocks, the full block $\mathcal{S}^\top$ is scheduled. Otherwise, the next subblock is pushed to $\mathcal{S}$ (becoming $\mathcal{S}^\top$) and scheduled.

After obtaining a block, it is processed using either Inc1S or Inc2S. There are two cases: (1) If the block is fully processed before reaching a timeout (cf. Algorithm 1, FullyProcessed), it is removed from $\mathcal{S}$ and the block's parents are updated. (2) Otherwise, the block was not fully processed before reaching a timeout (cf. Algorithm 1, PartiallyProcessed). This can only happen for the last block scheduled during a timeslice. In this case, the partially computed block is recursively split into its subblocks until the computed area of the partially computed block covers a subblock. Each subblock in the recursion is added to $\mathcal{S}$. Once these blocks reach the top of the stack again, their upper left subblock will have been computed already. Hence, we remove their UL subblock.

In the case of partially processed blocks, we discard the results from the partial block (cf. hatched area in Figure 3b) up to the last fully computed aligned subblock (resp. triangle, cf. shaded area in Figure 3b). We observe that the ratio of discarded results is bounded.

LEMMA 6.1. *At most 3/4 of the pairs of tuples processed in any timeslice are discarded upon reaching a timeout.*

PROOF. Consider the sequence of squares and triangles processed by any plan execution until reaching a timeout. We only discard result tuples of the last square (triangle). The largest aligned square (triangle) that was fully processed and is contained inside the last square (triangle) has size at least 1/4 of the next larger aligned square (triangle). Thus, at least 1/4 of all processed pairs of tuples are not discarded. $\square$

Example 6.2. Consider the block scheduling state depicted in Figure 5a. The UL subblock of B_1 was already fully processed. In the next step, B_2 is scheduled as it is the top of stack $\mathcal{S}$. Figure 5b shows the progress B_c of computing B_2 after reaching a timeout, i.e., B_2 is only partially processed. As B_2 is only partially processed, B_2 's UL subblock is partitioned two times until a resulting subblock is fully covered by B_c . The UL subblock of B_3 is fully processed, popped from $\mathcal{S}$, and B_4 is pushed to the stack to be scheduled as the next block.

Discussion. Discarding partially computed blocks simplifies the analysis and leads to at most constant multiplicative regret compared to the optimal algorithm choice by leveraging the bound on discarded result tuples. In practice, we continue the execution of Inc1S or Inc2S until the next larger subblock is fully processed. In those cases, the timeslice has varying size and the reward is the summed area over the measured time.

6.3 Regret Analysis

Assumptions. We analyze FAST with dynamic indexes (called DYN-FAST) under the assumption that the throughput of each query plan approximates a stationary distribution. In particular, the throughput does not change when changing the size of the timeslice. To this end, we select larger timeslices for DYNFAST than for STATFAST to reduce the skew introduced by disproportionately high indexing

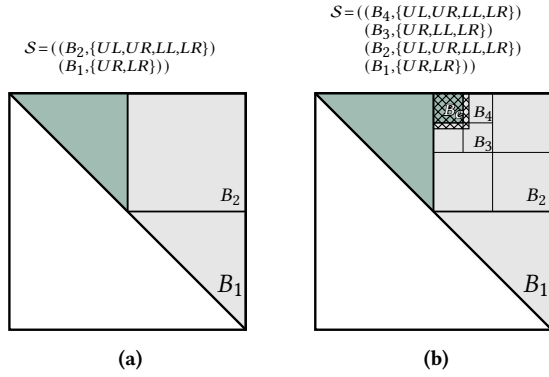


Figure 5: Example of Block Scheduling

and probing cost compared to the verification cost for small blocks. We will discuss the size of the timeslice in Section 7.

LEMMA 6.3. *DYNFAST has runtime regret of $(1 - 1/4)n + O(\log n)$.*

PROOF. We proceed as in Lemma 5.1. The main difference is that both R and R^* are not guaranteed to be one. By Lemma 6.1, at most $3/4$ of the achieved rewards are discarded. Hence, computation finishes after achieving at most a reward of 4. We have $1 \leq R, R^* \leq 4$. We can now bound the regret $n - n^*$ with $n(1 - \frac{R^*}{R}) = n(1 - \frac{rR^*}{rR}) \leq n(1 - \frac{r}{4r^*})$. As $r^* - r = O(\frac{\log n}{n})$ [39], we have $n(1 - \frac{r}{4r^*}) \leq n(1 - \frac{r^* - O(\frac{\log n}{n})}{4r^*})$ which is bounded by $(1 - \frac{1}{4})n + O(\log n)$. $\square$

LEMMA 6.4. *DYNFAST has bounded expected multiplicative runtime regret that converges to 4 with $n \rightarrow \infty$.*

PROOF. Like proof for Lemma 5.2. $\square$

6.4 Practical Considerations

Vectorization of Query Evaluation. In both STATFAST and DYNFAST as described in Sections 5.3 and 6.2, queries are evaluated record-by-record: A single record is reduced, probed, and candidates are verified. To reduce the overhead of performing multiple reductions, lookups inside an index, and one or more verification procedures, we partition all records into small *microbatches* and handle a microbatch of records in each invocation of a reduction, lookup, or verification. This style of *vectorized* evaluation of query plans is common in query engines [38].

Support for Index Updates. DYNFAST requires algorithms to support index updates in order to build indexes on-the-fly. In their original version, PREFIX [11, 14], PALLOC [18], PASSJOIN [41], and TJOIN [29] assume some order on the records to build their index. All algorithms require their records to be sorted in ascending size and possibly lexicographically based on their content to break ties. In general, different algorithms might require different and incompatible orderings, making it impossible to sort the data only once for all algorithms. Additionally, if an algorithm requires a reduction, we initially do not know which parts of the dataset will be reduced and need to be ordered. To this end, we adapt all algorithms to support incremental updates in any order.

All algorithms PREFIX, PALLOC, PASSJOIN, and TJOIN internally use a multi-level index structure. Some of those levels only perform point queries and some support range queries. As an example, a typical index implementing a prefix filter maps set elements to lists of record IDs sorted by record length. For a given set element (point query), the range of records possibly similar due to their size is scanned (range query). We replace each sorted list with an in-memory B^+ tree to support range queries and updates, while having high scan performance. All other algorithms are adopted similarly. In our implementation, we use `abs1::btree_map` from Abseil [1].

Caching of Indices. DYNFAST as described in Sections 6.2 evaluates a plan on a block by building and probing the respective index, which is discarded afterwards. We propose a caching strategy to reduce the overhead of index construction. The cache keeps a set of indexes for each plan and possibly reuses an index if the plan is selected again for a different block.

We reuse an index structure under the following conditions given some to-be-computed block B : (1) If some index fully covers either side of B , we select the smallest among them. We select the smallest index to reduce the number of required skips while scanning the index. (2) If an index according to (1) does not exist, we search for an index that starts at the same offset into the dataset as B . Among all of them, we select the largest index. Selecting the largest index reduces the overhead of otherwise having to probe multiple indexes. (3) Otherwise, use either INC1S or INC2S to build a new index. Due to the assumption on efficient range search (cf. Assumption (5), Section 6.1) cases (1) and (2) can be computed efficiently.

While current standalone implementations of similarity joins typically build their index on-the-fly [11, 18, 29, 70], database systems usually prebuild index structures and maintain them under updates [53]. As a significant side effect of implementing index reuse, DYNFAST supports using prebuilt indexes to potentially speed up query evaluation. To this end, any existing index for some query plan is added to the cache before execution of DYNFAST. During computation, DYNFAST can choose between using existing indexes or building a new index structure for some other query plan from scratch depending on the performance of each query plan.

Caching of Intermediary Reduction Data and Signatures. For DYNFAST, the same microbatch of records might appear in multiple partial join computations for the same query plan. To compute the partial join, each record in the microbatch is reduced, probed and/or indexed, and verified. To reduce the cost of reducing the same microbatch multiple times, we cache the result of the reduction. As multiple query plans might share some of their reduction steps, the result of each reduction step is cached to speed up the evaluation of related query plans. As an example, consider the query plans for TED in Figure 2. Assume that the query plan $TED \otimes SED \diamond Str.SS-Sym.SS \diamond$ is evaluated first. For the next timeslice, $TED \otimes SED \diamond$ is selected. Due to their common prefix, the cached reduction data of SED is reused. In this case, no further reductions are necessary. Otherwise, only the remaining reductions along the path are performed.

To reduce the cost of probing the same microbatch against the same index multiple times, we further cache intermediary data required to probe the index. In similarity search, most algorithms (and in particular all algorithms implemented in FAST) are based on a

signature scheme [52]. Intuitively, a signature scheme computes hash values for a given record. Depending on the algorithm, signature computation can be quite expensive. To reduce the burden of redundant signature computation for the same microbatch, we store its signatures in a cache. For our implementation, we use the *least recently used* replacement strategy for both caches.

7 EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of FAST. We first describe the datasets and the implementation of FAST that we use in our evaluation. We then study STATFAST’s effectiveness of finding good query plans by comparing STATFAST against all individual query plans. Finally, we compare STATFAST against DYNFAST and study both the scenario of a cold and a warm cache.

7.1 Experimental Setup

We conducted all experiments on a cluster of servers, each having one AMD EPYC 9354P CPU with 32 cores and 384 GiB memory. We executed between 1-4 jobs on each server in parallel, depending on required memory. We implemented FAST in C++. Missing data points correspond to timeouts (4 hours).

Datasets. We use twelve general and eight supplementary datasets in our evaluation. The general datasets are chosen from the pool of datasets used in previous works such that they cover diverse characteristics in document and universe size. The supplementary datasets are used to highlight specific aspects of an algorithm (e.g., due to skewed characteristics) and to reproduce experiments in Section 8. We refer to Mann *et al.* [44] for details on BMS-POS and Kosarak. DBLP14-Set² and Lnonis1 are described in Schmitt *et al.* [52]. All four general string datasets appeared in the empirical evaluation of string similarity joins by Jiang *et al.* [30]. All four general tree datasets are taken from Hütter *et al.* [29]. Gen20kl, Gen50ks, Trec₂, and Uniref have been used in recent works on approximate string edit distance joins and consist of longer strings [33, 73, 74]. JScript1k, Python1k, and Swissprot1k are subsets of larger tree datasets consisting only of large trees with 1000 nodes or more. These modified datasets first appeared in Karpov *et al.* [35]. SyntheticLU is a synthetic tree dataset with a small universe size; all trees have height four and fanout three; labels are uniformly distributed between 1 and 10000. Table 1 and 2 show characteristics³ of general and supplementary datasets, respectively.

Distance Functions. We evaluate FAST with Jaccard similarity on sets, the string edit distance on strings, and the tree edit distance on trees. We evaluated similar or larger ranges of distance thresholds ϵ compared to previous work. We study the extensibility of our approach to further distance functions in Section 9.

Algorithmic Parameters. We set all cache sizes (index, reduction, intermediate data) to $2|R|$. We use the reduction graph in Figure 1 without Euler strings due to their similarity to traversal strings. We set $q=3$ for the q -gram reduction. We set the timeslice to 0.2 seconds

²For DBLP14-Set, the dataset used in previous work is based on an older version of DBLP entries [52, 68]. Our dataset DBLP14-Set is based on the currently available version [59] and uses the same preprocessing steps as in [52, 68].

³Compared to previous work, we show the 99.9-th percentile instead of the the maximum set size. The maximum size is often not representative due to outliers.

Table 1: General dataset characteristics.

		#Documents	Document Size average 99.9 pct.		Universe
Sets	BMS-POS	$3.2 \cdot 10^5$	9.3	61	1657
	Kosarak	$6.1 \cdot 10^5$	11.9	378	41270
	DBLP14-Set	$5.2 \cdot 10^6$	77.7	209	24158
	Lnonis1	$8.2 \cdot 10^6$	20.3	39	44103
Strings	DBLP-String	$1.4 \cdot 10^6$	106.3	294	37
	Enron	$2.5 \cdot 10^5$	885.0	18332	37
	Trec	$3.5 \cdot 10^5$	845.2	2793	37
	Word	$1.2 \cdot 10^5$	9.7	19	26
Trees	DBLP-Tree	$3.9 \cdot 10^6$	26.1	81	$1.8 \cdot 10^7$
	Python	$1.5 \cdot 10^5$	944.1	23682	$3.4 \cdot 10^6$
	Sentiment	9645	37.3	99	19470
	Swissprot	$3.2 \cdot 10^5$	998.7	9561	$7.4 \cdot 10^6$

Table 2: Supplementary dataset characteristics.

		#Documents	Document Size average 99.9 pct.		Universe
Strings	Gen20kl	$2.0 \cdot 10^4$	20000	20058	5
	Gen50ks	$5.0 \cdot 10^4$	5000	5047	4
	Trec ₂	$2.3 \cdot 10^5$	1218.1	2834	37
	Uniref	$4.0 \cdot 10^5$	446	3519	24
Trees	JScript1k	$3.9 \cdot 10^4$	8775.8	$3.3 \cdot 10^5$	$2.5 \cdot 10^6$
	Python1k	$3.6 \cdot 10^4$	3012.6	$3.7 \cdot 10^4$	$2.6 \cdot 10^6$
	Swissprot1k	$1.2 \cdot 10^5$	1902.0	$13 \cdot 10^4$	$6.4 \cdot 10^6$
	SyntheticLU	$1.0 \cdot 10^6$	40	40	10000

for all datasets and STATFAST. In DYNFAST, we handle timeslices for Inc1S and Inc2S differently. As stated in Section 6, using larger timeslices reduces the skew introduced by disproportionately high indexing and probing cost. To this end, we set the timeslice to 0.2 for Inc1S and to 0.8 for Inc2S. Inspired by the scaling experiments in Trummer *et al.* [61], we set the exploration weight w (cf. Section 5.1) proportionally to the achieved rewards. While Trummer *et al.* [61] use a fixed weight, we dynamically update w at runtime. UCT requires random variables in $[0,1]$ and is often used with a weight of $\sqrt{2}$ [61]. In FAST, rewards represent relative throughput and tend to range between 0 and 10^{-4} . For such small rewards, high exploration weights lead to long convergence times. To this end, we scale the exploration weight depending on the observed rewards: Initially, we set $w=0.01\sqrt{2}$ (corresponding to an assumed upper bound of 10^{-2} on the rewards). During execution, we keep track of the maximum observed reward r_{max} and update $w = \sqrt{2}r_{max}$ with exponentially increasing intervals between each update.

7.2 Evaluation of STATFAST

We evaluate the ability of STATFAST to select highly performant query plans. To this end, we compare STATFAST against a standalone execution of each individual query plan. In this setting, we assume that all indexes are prebuilt and only measure the time to compute the join. Figure 6 reports the results on all general datasets. A query plan is named after its reduction step(s) (Q: q -grams, T: traversal strings, L: label sets) and the join algorithm.

In general, the best query plan differs for different datasets and thresholds. On set data, PALLO performs best in most settings, only being outperformed by PREFIX on high thresholds in Kosarak. The

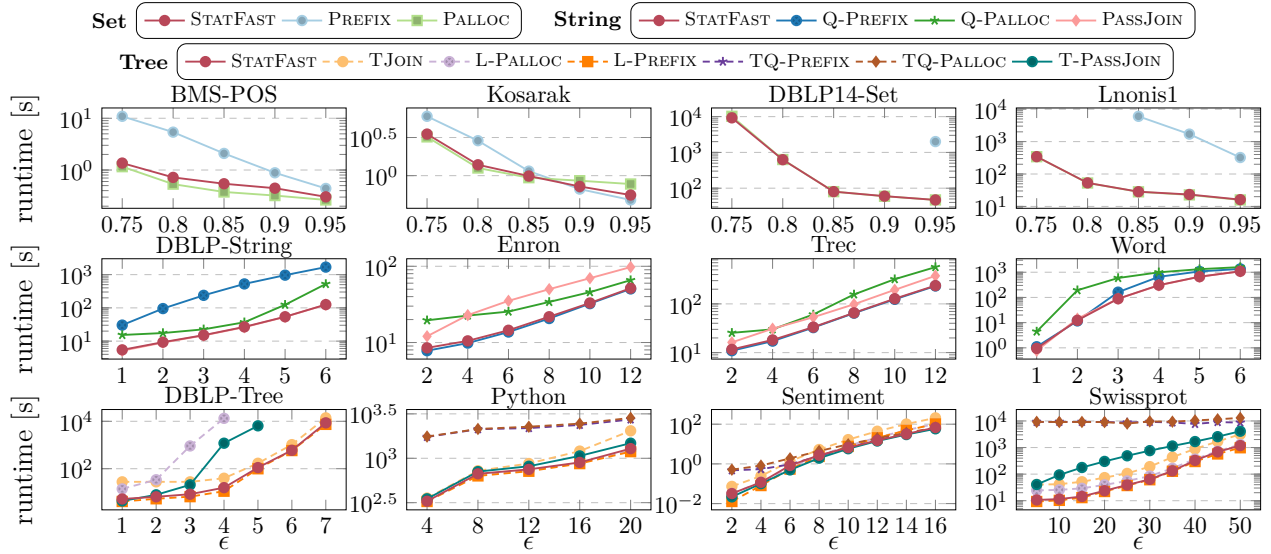


Figure 6: Comparison of STATFAST against all individual query plans.

prefix filter requires set elements to be sorted in inverse global frequency for high performance, while PALLOC needs no preprocessing. Compared to most previous work [44], we do not assume that the input data is already preprocessed. Therefore, PREFIX pays the extra cost of computing a new set element mapping.

On strings, PASSJOIN and Q-PREFIX compete for the best query plan. On datasets with short strings (DBLP-String and Word), PASSJOIN is the most selective query plan and requires few verifications. For longer strings and higher thresholds (Enron and Trec), the lightweight Q-PREFIX approach is selective enough.

For trees, the label-set based query plans and in particular L-PREFIX are the fastest on all datasets except Sentiment due to their low overhead. This is due to the large universe size; many uncommon tokens are readily leveraged by PREFIX and PALLOC to filter candidates without having to consider the actual tree structure. Sentiment has a smaller universe size than other datasets, requiring methods that preserve more of the tree structure than label sets. Hence, traversal-string based approaches (in particular T-PASSJOIN) slightly outperform the other plans. Motivated by this observation, we additionally evaluate STATFAST on the supplementary dataset SyntheticLU (not shown) to compare the performance of query plans on a dataset with a small universe size. On that dataset, plans based on the prefix filter (PREFIX and TJoin) are outperformed by the more selective T-PASSJOIN and L-PALOC by up to three orders of magnitude.

STATFAST closely follows the best performing query plan. On average over all thresholds and datasets, STATFAST is only 12% slower than the fastest query plan. The worst case happens for extremely short runtimes on the Sentiment dataset at threshold $\epsilon = 2$, where STATFAST requires 0.03 seconds compared to the optimal 0.012 seconds (factor 2.5). When only considering runtimes of at least 1 second, the average and maximum slowdown of STATFAST shrink to 7% and 38%, respectively. This shows that STATFAST effectively selects the fastest query plan and has little overhead.

7.3 Evaluation of DYNFAST

We now compare the performance of DYNFAST and STATFAST. To this end, we consider two scenarios for each algorithm: (1) **Cold**: STATFAST (from scratch) is executed without prebuilt indexes; indexes must be constructed before executing the join. DYNFAST (cold) is executed starting with empty index, reduction, and signature caches. (2) **Warm**: For STATFAST (prebuilt), all indexes are built before query evaluation; they do not have to be constructed before executing the join. DYNFAST (warm) is executed with warm index, reduction, and signature caches. The caches are warmed up by executing the same join twice and only measuring the second execution.

Runtime Evaluation. Figure 7 shows the runtime performance of all four scenarios. We expect the following descending order of performance for the different scenarios: (1) STATFAST (prebuilt) does not require indexing and has less overhead than DYNFAST. (2) DYNFAST (warm) can leverage its caches to avoid most index construction work and reduce the overhead of DYNFAST. (3) DYNFAST (cold) starts with an empty index cache, but does not construct all indexes. (4) STATFAST (from scratch) constructs all indexes and is unsuitable if the number of query plans is high.

The results in Figure 7 are consistent with our expectation for most datasets (Kosarak, Word, DBLP-Tree, Python, Sentiment, Swissprot, high ϵ for DBLP-String, Enron, and Trec). For low distance thresholds ϵ , DYNFAST (warm) outperforms STATFAST (prebuilt) on BMS-POS (similarity $\epsilon \geq 0.75$), DBLP-String ($\epsilon \leq 2$), Enron ($\epsilon \leq 6$), and Trec ($\epsilon \leq 4$). This result is an effect of caching at the hardware level: For low thresholds, indexes are small and might fit into the processor’s cache. In DYNFAST (warm), we execute the same query twice and thus access the same index structures that are already in the processor cache. When flushing the hardware cache in between runs, the difference between DYNFAST (warm) and STATFAST (prebuilt) decreases for all results affected by this phenomenon.

On DBLP-14 and Lnonis1, for high thresholds, probing is expensive compared to verification. In general, DYNFAST requires probing

the same record multiple times against small, partial indexes. Even with caching of intermediary reduction data and signatures, DYNFAST falls behind STATFAST on these datasets.

On average over all datasets and thresholds, DYNFAST (warm), DYNFAST (cold), and STATFAST (from scratch) are 58%, 197%, and 306% slower than STATFAST (prebuilt).

Evaluation of Index Redundancy. We further analyze the *index redundancy* of DYNFAST compared to STATFAST. We define the index redundancy as the average number of *additional indexes* that a single record is indexed as compared to being only indexed in a single index. Table 3 lists the average index redundancy of DYNFAST and the relative reduction to STATFAST on all datasets. For STATFAST, each query plan corresponds to an index and thus its index redundancy is $|\mathcal{P}| - 1$. DYNFAST avoids building major parts of inefficient indexes, only building highly performant ones. On average, DYNFAST requires 80%, 77%, and 82% fewer additional indexes on sets, strings, and trees, respectively.

In summary, our results show that (1) DYNFAST (warm) requires significantly fewer redundant indexes, but has a moderate runtime penalty compared to STATFAST (prebuilt) due to its probing overhead, and (2) DYNFAST (cold) outperforms STATFAST (from scratch) both in terms of runtime and index redundancy.

8 FAST AS A BASELINE

In this section, we evaluate the performance of FAST as a baseline for new join algorithms. We consider three recently proposed algorithms for sets, strings, and trees and compare them to FAST. We further discuss if the inclusion of the respective algorithm into FAST can lead to a significantly better query plans.

8.1 Comparison against TwoL (Sets)

Introduction. TwoL [52] was recently proposed as a similarity join algorithm on sets for symmetric set similarity functions. Its main contribution is its ability to combine two index structures for set similarity and adapt to the dataset’s characteristics. Compared to FAST, it only supports similarity search on sets and is based on optimizing a cost model estimating the cost of a join using either parts of individual indexes or both at the same time. It promises high performance on datasets with mixed characteristics.

Setup. We compare the author’s implementation of TwoL against STATFAST (from scratch) on all set datasets (cf. Table 1). Compared to FAST, TwoL requires preprocessed datasets. The preprocessing time is not included in our results. We compare on Jaccard similarity. For Jaccard, TwoL combines PREFIX with PALLOC. Similarly, FAST can select the PREFIX or the PALLOC plan.

Experimental Results. Figure 8 shows the runtime (left axis) of TwoL and STATFAST (from scratch); PREFIX and PALLOC are included for reference. TwoL and FAST perform nearly identically. They both effectively avoid the highly inefficient choice of PREFIX on DBLP14-Set and Lnonis1. The right axis of Figure 8 shows the usage ratio of PALLOC (# timeslices for FAST, # reindexed records for TwoL) in TwoL and FAST. Both algorithms are clearly correlated in their choices.

Table 3: Index Redundancy of STATFAST vs. DYNFAST.

Dataset	redund.	% red.	Dataset	redund.	% red.
BMS-POS	0	−100%	Trec	0.32	−84%
Kosarak	0.60	−40%	Word	1.04	−48%
DBLP14-Set	0.14	−86%	DBLP-Tree	0.47	−91%
Lnonis1	0.08	−92%	Python	1.7	−66%
DBLP-String	0.16	−92%	Sentiment	0.95	−81%
Enron	0.30	−85%	Swissprot	0.55	−89%

Conclusion. FAST behaves nearly identically to TwoL both in terms of runtime and index decisions. Hence, including TwoL into FAST will not yield any significant improvement.

8.2 Comparison against MINJOIN++ (Strings)

Introduction. MINJOIN++ [33] is an approximate algorithm for the string edit distance. It is based on splitting all strings into disjoint substrings using pivot points selected with random hash functions on q -grams. Compared to other algorithms like PASSJOIN, the number of substrings is linear in ϵ . MINJOIN++ focuses on long strings and promises finding all pairs with high probability.

Setup. We compare MINJOIN++ against FAST on all four general string datasets (cf. Table 1) and four supplementary string datasets (Gen20kl, Gen50ks, Trec₂, Uniref; cf. Table 2) that were used in the original evaluation of MINJOIN++ [33]. We use the author’s implementation of MINJOIN++ [34] and set all parameters according to the authors default suggestions [33]. In addition to 3-grams included in FAST by default (cf. Section 7), we add additional q -gram reductions for $q \in \{5, 7, 9, 11, 13, 15\}$ due to the small universe sizes of the genetic datasets (Gen20kl, Gen50ks).

Experimental Results. Figure 9 shows the runtime of MINJOIN++ and STATFAST (prebuilt) on DBLP-String, Word, GEN20kl, and Uniref. The performance on Enron and Trec (not shown) is similar to Word, while the performance on Gen50ks and Trec₂ (not shown) is similar to Gen20kl and Uniref, respectively. For MINJOIN++, we additionally show the recall for each data point. On datasets with very long strings and high thresholds (Gen20kl, Gen50ks, Trec₂, Uniref), MINJOIN++ significantly outperforms FAST while achieving high recall (>99%). For datasets with short strings (Word, DBLP-String) and for larger thresholds on Enron and Trec, MINJOIN++ only achieves low recalls (<50%) in many cases. This is a known limitation of MINJOIN++ due to its use of randomization [33].

Conclusion. The performance of MINJOIN++ heavily depends on the dataset. On datasets with short strings, MINJOIN++ achieves low recall and high runtimes; on datasets with long strings, MINJOIN++ finds all result pairs with a significant speedup compared to FAST. Therefore, we consider MINJOIN++ a worthwhile candidate for inclusion in the reduction graph of FAST.

8.3 Comparison against SYNC SIG (Trees)

Introduction. SYNC SIG [35] introduces two approximate algorithms (BJOIN and EJOIN) for TED. Both are similar to MINJOIN and either partition the trees into balls (BJOIN) or convert the trees into Euler strings and partition the strings (EJOIN). Similar to MINJOIN, both algorithms claim high performance for large trees.

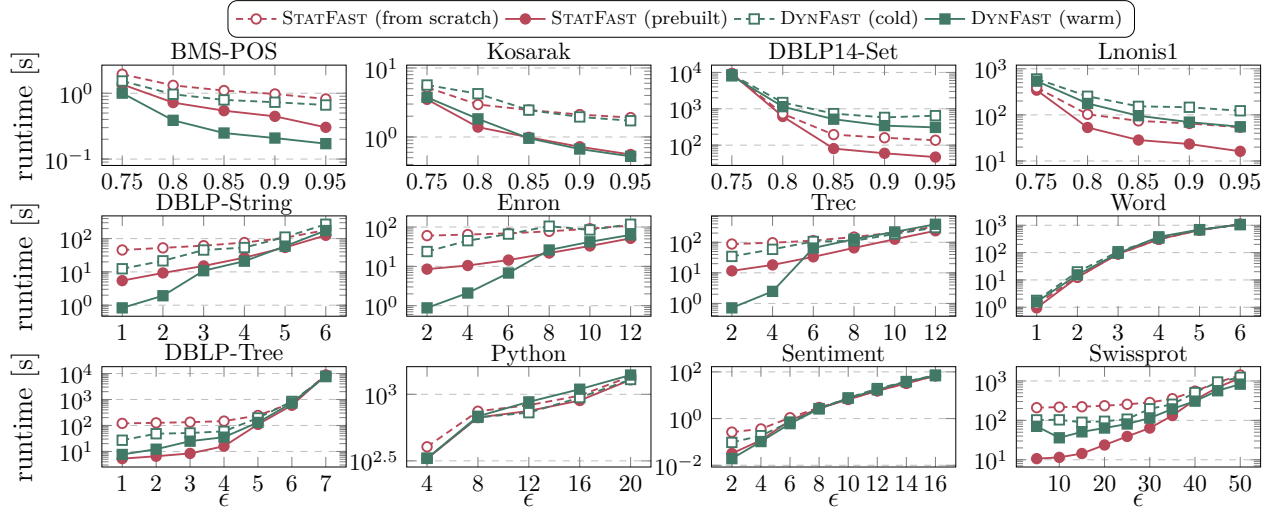


Figure 7: Comparison of STATFAST and DYNFAST.

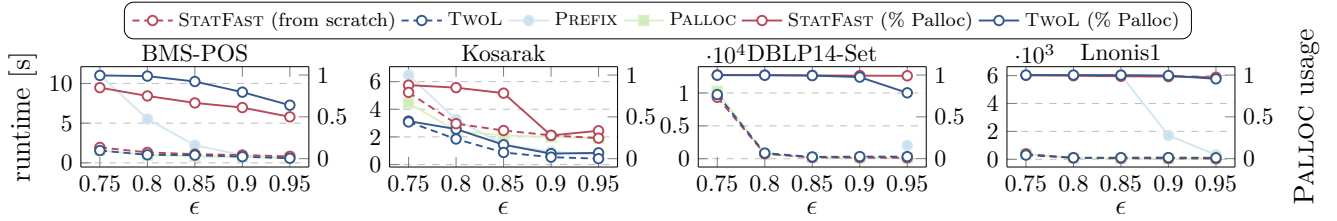


Figure 8: Runtime comparison of FAST against TwoL.

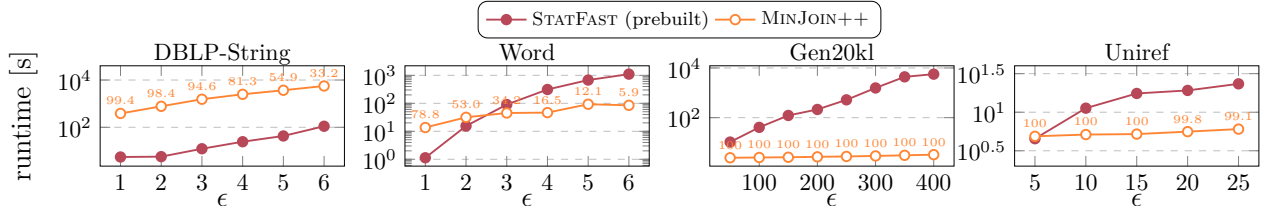


Figure 9: Runtime comparison of FAST against MINJOIN++.

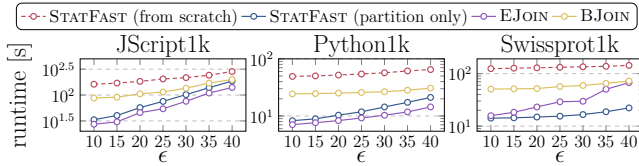


Figure 10: Runtime comparison of FAST against SYNC SIG.

Setup. SYNC SIG was already compared to TJOIN on the general datasets (cf. Table 1) in the original work [35]. For these datasets, no significant difference between all three algorithms was found: They all must verify at least the true-positive trees and the verification step is responsible for most of the runtime. The authors thus remove all trees smaller than 1000 nodes to reduce the output size. We use their datasets (JScript1k, Python1k, Swissprot1k; cf. Table 2) for this evaluation. We use the author’s implementation of SYNC SIG [36].

Experimental Results. Figure 10 shows the runtime of SYNC SIG in both its BJOIN and EJOIN variant and STATFAST (from scratch, i.e., including indexing time). After the execution of STATFAST, we observe that the query plan $TED \blacklozenge Str.SS-Sym.SS-\blacklozenge^4$ outperforms all other query plans significantly. We then measure only the performance of this single query plan (referred to as *STATFAST (partition only)* in Figure 10). STATFAST (partition only) always outperforms BJOIN and shows similar performance to EJOIN. The remaining difference in runtime on JScript1k and Python1k can be attributed to the materialization of data reductions: By computing PALOC directly on the nodes of the tree without materializing them as sets, this gap can be reduced.

Conclusion. To the best of our knowledge, no existing algorithm for TED has used a query plan similar to the fastest one based on PALOC generated by FAST. This query plan is efficient and performs approximately as well as the better variation of SYNC SIG on all tested

⁴The variation of PALOC used here disables deletion neighborhoods and is equivalent to the partition filter in PARTENUM [4].

datasets. Hence, the addition of SYNC SIG to FAST would not yield a significant improvement.

9 FAST AS A TOOL FOR PROTOTYPING

We show the extensibility of FAST to an unsupported similarity function as a method to design a similarity join algorithm. To this end, we demonstrate the inclusion of the Jaro similarity on strings and show the required steps to implement baseline support for Jaro inside FAST using reductions. Furthermore, we compare FAST against LIMES [21, 47], a state-of-the-art algorithm for Jaro similarity joins.

Jaro Similarity. The Jaro similarity is a measure defined on strings. For two strings r, s , we define the number of matches m as the number of identical characters that appear at most $\frac{\max\{|r|, |s|\}}{2}$ character positions apart from another in r and s . Furthermore, we define the number of transpositions t as the cardinality of the subset of matches that are not matched at the same position in both strings. Then, the Jaro similarity is defined as $d_j(r, s) = \frac{1}{3} (\frac{m}{|r|} + \frac{m}{|s|} + \frac{m-t}{m})$. Jaro similarity is used in entity linking and related fields [16, 21, 49].

Implementing Jaro using Reductions. Besides an implementation of the similarity function itself, a reduction to a known distance function with known query plans is required. To this end, we use a common bound of d_j based on the overlap of the character sets of two strings [21, 66]: Define *Jaro Overlap* similarity as $d_{jo} = \frac{1}{3} (\frac{|r \cap s|}{|r|} + \frac{|r \cap s|}{|s|} + 1)$, where the strings r, s are interpreted as sets of characters. Then $d_j(r, s) \leq d_{jo}(r, s)$. Jaro Overlap is an instance of a symmetric set similarity, with the equivalent overlap defined as $eq_{jo}(r, s, \epsilon) = (3\epsilon - 1) \frac{|r \cap s|}{|r| + |s|}$. Thus, Jaro Overlap does not require any significant implementation effort as it is defined by its equivalent overlap like all other symmetric set similarities.

To summarize, implementing Jaro similarity inside FAST only requires (1) implementing Jaro similarity for pairs of strings, (2) adding a new equivalent overlap function to support Jaro Overlap, (3) implementing the *character sets* reduction, mapping strings to sets (implemented as a special case of q -grams with $q=1$), and (4) registering Jaro, Jaro Overlap, and character sets in the reduction graph (cf. Figure 1). Our reduction graph finds two query plans for Jaro similarity: Jaro is reduced to Jaro Overlap using character sets; then, Jaro Overlap is evaluated using PREFIX or PALLOC.

Experimental Evaluation. We compare FAST against LIMES [21, 47], a state-of-the-art algorithm for Jaro similarity joins. We adapt the original implementation of LIMES [22]: (1) We reimplement the original Java implementation of LIMES in C++, (2) adjusted an incorrect early termination condition that was previously noticed by Keil *et al.* [37], and (3) removed the use of redundant datastructures. Except for modification (2) that is required for correctness, all modifications increased the performance of LIMES.

We evaluate FAST and LIMES on all general datasets. Due to high runtimes of low Jaro thresholds on these datasets, we take samples such that the runtime of LIMES does not significantly exceed one hour. For DBLP-string, Enron, and Trec, the samples are of size 60000, 11000, and 8000, respectively. We use the full Word dataset.

Figure 11 shows the runtime of LIMES and STATFAST (from scratch). Except for low thresholds on Enron and Trec, STATFAST significantly outperforms LIMES for all datasets. LIMES essentially compares all

pairs of strings, but reduces the verification cost using early termination and by verifying groups of strings that share characters at the same time. FAST leverages both PREFIX and PALLOC to effectively prune dissimilar pairs and thus does not need to consider all pairs of strings. In particular, FAST tends to use PREFIX for filtering of low thresholds (up to 0.88–0.94, depending on dataset) and PALLOC for high thresholds (0.88–0.94 and higher).

Conclusion of Case Study. Extending FAST to Jaro similarity only requires adding the Jaro similarity function, an equivalent overlap function, a reduction, and new entries in the reduction graph. In particular, no new algorithms are required. This prototype serves as a baseline for algorithms using character sets to bound Jaro similarity. FAST outperforms the current state-of-the-art algorithm for Jaro joins LIMES by one to three orders of magnitude in most settings.

10 RELATED WORK

Standalone Algorithms. A large number of standalone algorithms for similarity joins has been proposed in literature. These works are orthogonal to our work as, to the best of our knowledge, all current algorithms can be included into FAST to extend it with new query plans.

Most algorithms for set similarity joins either use the prefix filter [14] or the partition-and-enumeration framework [4]. We refer to the survey by Mann *et al.* [44] for details on the numerous variations of prefix-filter based algorithms [11, 13, 14, 43, 63, 71]. Further developments include grouping of index entries and probing signatures [65], efficiently enumerating subsets as signatures [19], and techniques to combine multiple algorithms depending on the dataset [52, 68].

For string similarity joins, most algorithms focus on the string edit distance and we refer to Jiang *et al.* [30] for details. Recent developments for approximate string similarity joins include embedding SED in Hamming space [73] and splitting the strings at pivot points determined by a randomized function [33, 74].

In the context of tree edit distance joins, Li *et al.* [40] survey bounds based on histograms [32], strings [27], and subtrees [72]. Further developments include splitting the trees into subgraphs and applying the pigeonhole principle [60], using enhanced label sets including positional information [29], and approximate approaches based on tree partitioning or Euler string partitioning [35].

Similarity Joins in Systems. Recently, some systems taking a more holistic approach to similarity joins have been proposed. SIREN [10] is built on top of a traditional DBMS and handles similarity queries externally. DIMA [57, 58] implements PALLOC [18] in Apache Spark and extends it for distributed computation. DIMA performs cost-based optimization either using a prebuilt index or by sampling. Silva *et al.* [54] study different similarity operators, their conceptual evaluation, and rewrite rules for query optimization. Wang *et al.* [67] use machine learning for cardinality estimation of similarity joins. Sun *et al.* [56] reduce the dimensionality of their training data by partitioning and learning local models. Different from previous work, we focus on an extensible system for different similarity functions and domains. Note that selecting an efficient query plan is orthogonal to estimating the cardinality of the result of the similarity join. While all joins have the same cardinality, different similarity join algorithms greatly vary in cost.

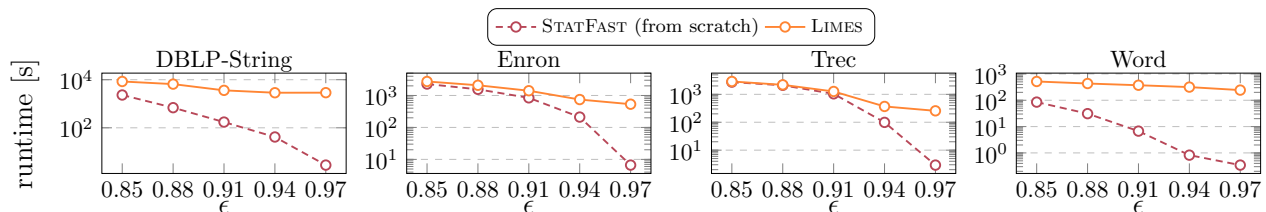


Figure 11: Runtime comparison of FAST against LIMES.

Feedback-based Query Evaluation. Several query plan selection schemes using measured runtime performance have been proposed recently. Eddies [9] routes input tuples to the plan's operators based on lottery scheduling [62] without formal guarantees. Li *et al.* [42] dynamically change join orders depending on observed selectivities. Trummer *et al.* [61] use UCT [39] to find efficient join orders during execution. Compared to Trummer, our work on similarity queries introduces a novel technique for query plan enumeration, requires different MDP modelling, a new analysis, and different strategies for work sharing.

11 CONCLUSION

We introduced FAST, a similarity join algorithm that is both extensible and efficient, overcoming limitations of current monolithic solutions. By leveraging reductions to transform complex join problems into simpler ones, our system efficiently enumerates and selects optimal query plans via a sampling-based strategy without the need for cost models. Supporting both prebuilt and on-the-fly indexing with effective caching, FAST achieves performance near the best individual plan in our extensive evaluation. Finally, our case studies further demonstrate FAST's extensibility and potential as a baseline.

REFERENCES

- [1] 2017–2025. Abseil Common Libraries. <https://abseil.io/>.
- [2] Tatsuya Akutsu. 2006. A relation between edit distance for ordered trees and edit distance for Euler strings. *Inf. Process. Lett.* 100, 3 (2006), 105–109. <https://doi.org/10.1016/j.ipl.2006.06.002>
- [3] Tatsuya Akutsu, Daiji Fukagawa, and Atsuhiko Takasu. 2010. Approximating Tree Edit Distance through String Edit Distance. *Algorithmica* 57, 2 (2010), 325–348. <https://doi.org/10.1007/s00453-008-9213-z>
- [4] Arvind Arasu, Venkatesh Ganti, and Raghav Kaushik. 2006. Efficient Exact Set-Similarity Joins. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12–15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 918–929. <http://dl.acm.org/citation.cfm?id=1164206>
- [5] Nikolaus Augsten. 2018. A Roadmap towards Declarative Similarity Queries. In *Proceedings of the 21st International Conference on Extending Database Technology, EDBT 2018, Vienna, Austria, March 26–29, 2018*, Michael H. Böhlen, Reinhard Pichler, Norman May, Erhard Rahm, Shan-Hung Wu, and Katja Hose (Eds.). OpenProceedings.org, 509–512. <https://doi.org/10.5441/002/EDBT.2018.59>
- [6] Nikolaus Augsten and Michael H. Böhlen. 2013. *Similarity Joins in Relational Database Systems*. Morgan & Claypool Publishers. <https://doi.org/10.2200/S00544ED1V01Y201310DTM038>
- [7] Nikolaus Augsten, Michael H. Böhlen, and Johann Gamper. 2005. Approximate Matching of Hierarchical Data Using pq-Grams. In *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 – September 2, 2005*, Klemens Böhm, Christian S. Jensen, Laura M. Haas, Martin L. Kersten, Per-Ake Larson, and Beng Chin Ooi (Eds.). ACM, 301–312. <http://www.vldb.org/archives/website/2005/program/paper/wed/p301-augsten.pdf>
- [8] Nikolaus Augsten, Michael H. Böhlen, and Johann Gamper. 2010. The pq-gram distance between ordered labeled trees. *ACM Trans. Database Syst.* 35, 1 (2010), 4:1–4:36. <https://doi.org/10.1145/1670243.1670247>
- [9] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16–18, 2000, Dallas, Texas, USA*, Weidong

- Chen, Jeffrey F. Naughton, and Philip A. Bernstein (Eds.). ACM, 261–272. <https://doi.org/10.1145/342009.335420>
- [10] Maria Camila Nardini Barioni, Humberto Luiz Razente, Agma J. M. Traina, and Caetano Traina Jr. 2006. SIREN: A Similarity Retrieval Engine for Complex Data. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12–15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 1155–1158. <http://dl.acm.org/citation.cfm?id=1164232>
- [11] Roberto J. Bayardo, Yiming Ma, and Ramakrishnan Srikant. 2007. Scaling up all pairs similarity search. In *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada, May 8–12, 2007*, Carey L. Williamson, Mary Ellen Zurko, Peter F. Patel-Schneider, and Prashant J. Shenoy (Eds.). ACM, 131–140. <https://doi.org/10.1145/1242572.1242591>
- [12] Thomas Bocek, Ela Hunt, David Hausheer, and Burkhard Stiller. 2008. Fast similarity search in peer-to-peer networks. In *IEEE/IFIP Network Operations and Management Symposium: Pervasive Management for Ubiquitous Networks and Services, NOMS 2008, 7–11 April 2008, Salvador, Bahia, Brazil*, Marcus Brunner, Carlos Becker Westphall, and Lisandro Zambenedetti Granville (Eds.). IEEE, 240–247. <https://doi.org/10.1109/NOMS.2008.4575140>
- [13] Panagiotis Boursos, Shen Ge, and Nikos Mamoulis. 2012. Spatio-textual similarity joins. *Proc. VLDB Endow.* 6, 1 (2012), 1–12. <https://doi.org/10.14778/2428536.2428537>
- [14] Surajit Chaudhuri, Venkatesh Ganti, and Raghav Kaushik. 2006. A Primitive Operator for Similarity Joins in Data Cleaning. In *Proceedings of the 22nd International Conference on Data Engineering, ICDE 2006, 3–8 April 2006, Atlanta, GA, USA*, Ling Liu, Andreas Reuter, Kyu-Young Whang, and Jianjun Zhang (Eds.). IEEE Computer Society, 5. <https://doi.org/10.1109/ICDE.2006.9>
- [15] Tobias Christiani and Rasmus Pagh. 2017. Set similarity search beyond MinHash. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, Montreal, QC, Canada, June 19–23, 2017*, Hamed Hatami, Pierre McKenzie, and Valerie King (Eds.). ACM, 1094–1107. <https://doi.org/10.1145/3055399.3055443>
- [16] William W. Cohen, Pradeep Ravikumar, and Stephen E. Fienberg. 2003. A Comparison of String Distance Metrics for Name-Matching Tasks. In *Proceedings of IJCAI-03 Workshop on Information Integration on the Web (IIWeb-03), August 9–10, 2003, Acapulco, Mexico*, Subbarao Kambhampati and Craig A. Knoblock (Eds.). 73–78. <http://www.isi.edu/info-agents/workshops/ijcai03/papers/Cohen-p.pdf>
- [17] Pierre-Arnaud Coquelin and Rémi Munos. 2007. Bandit Algorithms for Tree Search. In *UAI 2007, Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence, Vancouver, BC, Canada, July 19–22, 2007*, Ronald Parr and Linda C. van der Gaag (Eds.). AUAI Press, 67–74. <https://doi.org/10.5555/3020488.3020497>
- [18] Dong Deng, Guoliang Li, He Wen, and Jianhua Feng. 2015. An Efficient Partition Based Method for Exact Set Similarity Joins. *Proc. VLDB Endow.* 9, 4 (2015), 360–371. <https://doi.org/10.14778/2856318.2856330>
- [19] Dong Deng, Yufei Tao, and Guoliang Li. 2018. Overlap Set Similarity Joins with Theoretical Guarantees. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10–15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 905–920. <https://doi.org/10.1145/3183713.3183748>
- [20] Carmel Domshlak and Zohar Feldman. 2013. To UCT, or not to UCT? (Position Paper). In *Proceedings of the Sixth Annual Symposium on Combinatorial Search, SOCS 2013, Leavenworth, Washington, USA, July 11–13, 2013*, Malte Helmert and Gabriele Röger (Eds.). AAAI Press, 63–70. <https://doi.org/10.1609/SOCS.V4I1.18299>
- [21] Kevin Dreßler and Axel-Cyrille Ngonga Ngomo. 2017. On the efficient execution of bounded Jaro-Winkler distances. *Semantic Web* 8, 2 (2017), 185–196. <https://doi.org/10.3233/SW-150209>
- [22] Kevin Dreßler and Axel-Cyrille Ngonga Ngomo. 2017. On the efficient execution of bounded Jaro-Winkler distances (Source Code). <https://github.com/dice-group/LIMES>. Source code accompanying the paper.
- [23] Zohar Feldman and Carmel Domshlak. 2013. Monte-Carlo Planning: Theoretically Fast Convergence Meets Practical Efficiency. In *Proceedings of the Twenty-Ninth Conference on Uncertainty in Artificial Intelligence, UAI 2013, Bellevue, WA, USA, August 11–15, 2013*, Ann E. Nicholson and Padhraic Smyth (Eds.). AUAI Press. https://dslpitt.org/uai/displayArticleDetails.jsp?mmnu=1&smnu=2&article_id=2382&proceeding_id=29

- [24] Zohar Feldman and Carmel Domshlak. 2014. Simple Regret Optimization in Online Planning for Markov Decision Processes. *J. Artif. Intell. Res.* 51 (2014), 165–205. <https://doi.org/10.1613/JAIR.4432>
- [25] Jianhua Feng, Jiannan Wang, and Guoliang Li. 2012. Trie-join: a trie-based method for efficient string similarity joins. *VLDB J.* 21, 4 (2012), 437–461. <https://doi.org/10.1007/S00778-011-0252-8>
- [26] Luis Gravano, Panagiotis G. Ipeirotis, H. V. Jagadish, Nick Koudas, S. Muthukrishnan, and Divesh Srivastava. 2001. Approximate String Joins in a Database (Almost) for Free. In *VLDB 2001, Proceedings of 27th International Conference on Very Large Data Bases, September 11–14, 2001, Roma, Italy*, Peter M. G. Apers, Paolo Atzeni, Stefano Ceri, Stefano Paraboschi, Kotagiri Ramamohanarao, and Richard T. Snodgrass (Eds.). Morgan Kaufmann, 491–500. <http://www.vldb.org/conf/2001/P491.pdf>
- [27] Sudipto Guha, H. V. Jagadish, Nick Koudas, Divesh Srivastava, and Ting Yu. 2002. Approximate XML joins. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, Madison, Wisconsin, USA, June 3–6, 2002*, Michael J. Franklin, Bongki Moon, and Anastasia Ailamaki (Eds.). ACM, 287–298. <https://doi.org/10.1145/564691.564725>
- [28] Thomas Hütter, Nikolaus Augsten, Christoph M. Kirsch, Michael J. Carey, and Chen Li. 2022. JEDI: These aren't the JSON documents you're looking for?. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12–17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 1584–1597. <https://doi.org/10.1145/3514221.3517850>
- [29] Thomas Hütter, Mateusz Pawlik, Robert Loschinger, and Nikolaus Augsten. 2019. Effective Filters and Linear Time Verification for Tree Similarity Joins. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8–11, 2019*, IEEE, 854–865. <https://doi.org/10.1109/ICDE.2019.00081>
- [30] Yu Jiang, Guoliang Li, Jianhua Feng, and Wen-Syan Li. 2014. String Similarity Joins: An Experimental Evaluation. *Proc. VLDB Endow.* 7, 8 (2014), 625–636. <https://doi.org/10.14778/2732296.2732299>
- [31] Liang Jin, Chen Li, and Rares Vernica. 2008. SEPIA: estimating selectivities of approximate string predicates in large Databases. *VLDB J.* 17, 5 (2008), 1213–1229. <https://doi.org/10.1007/S00778-007-0061-2>
- [32] Karin Kailing, Hans-Peter Kriegel, Stefan Schöner, and Thomas Seidl. 2004. Efficient Similarity Search for Hierarchical Data in Large Databases. In *Advances in Database Technology - EDBT 2004, 9th International Conference on Extending Database Technology, Heraklion, Crete, Greece, March 14–18, 2004, Proceedings (Lecture Notes in Computer Science)*, Elisa Bertino, Stavros Christodoulakis, Dimitris Plexousakis, Vassilis Christophides, Manolis Koubarakis, Klemens Böhm, and Elena Ferrari (Eds.), Vol. 2992. Springer, 676–693. https://doi.org/10.1007/978-3-540-24741-8_39
- [33] Nikolai Karpov, Haoyu Zhang, and Qin Zhang. 2024. MinJoin++: a fast algorithm for string similarity joins under edit distance. *VLDB J.* 33, 2 (2024), 281–299. <https://doi.org/10.1007/S00778-023-00806-Z>
- [34] Nikolai Karpov, Haoyu Zhang, and Qin Zhang. 2024. MinJoin++: a fast algorithm for string similarity joins under edit distance (Source Code). <https://github.com/kedayuge/MinJoin>. Source code accompanying the paper.
- [35] Nikolai Karpov and Qin Zhang. 2022. SyncSignature: A Simple, Efficient, Parallelizable Framework for Tree Similarity Joins. *Proc. VLDB Endow.* 16, 2 (2022), 330–342. <https://doi.org/10.14778/3565816.3565833>
- [36] Nikolai Karpov and Qin Zhang. 2022. SyncSignature: A Simple, Efficient, Parallelizable Framework for Tree Similarity Joins (Source Code). <https://github.com/nkarpov/syncsignature>. Source code accompanying the paper.
- [37] Jan Martin Keil. 2019. Efficient Bounded Jaro-Winkler Similarity Based Search. In *Datenbanksysteme für Business, Technologie und Web (BTW 2019), 18. Fachtagung des GI-Fachbereichs „Datenbanken und Informationssysteme“ (DBIS), 4–8. März 2019, Rostock, Germany, Proceedings (LNI)*, Torsten Grust, Felix Naumann, Alexander Böhm, Wolfgang Lehner, Theo Härder, Erhard Rahm, Andreas Heuer, Meike Klettke, and Holger Meyer (Eds.), Vol. P-289. Gesellschaft für Informatik, Bonn, 205–214. <https://doi.org/10.18420/BTW2019-13>
- [38] Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, and Peter A. Boncz. 2018. Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask. *Proc. VLDB Endow.* 11, 13 (2018), 2209–2222. <https://doi.org/10.14778/3275366.3275370>
- [39] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006, 17th European Conference on Machine Learning, Berlin, Germany, September 18–22, 2006, Proceedings (Lecture Notes in Computer Science)*, Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou (Eds.), Vol. 4212. Springer, 282–293. https://doi.org/10.1007/11871842_29
- [40] Fei Li, Hongzhi Wang, Jianzhong Li, and Hong Gao. 2013. A survey on tree edit distance lower bound estimation techniques for similarity join on XML data. *SIGMOD Rec.* 42, 4 (2013), 29–39. <https://doi.org/10.1145/2590989.2590994>
- [41] Guoliang Li, Dong Deng, Jiannan Wang, and Jianhua Feng. 2011. PASS-JOIN: A Partition-based Method for Similarity Joins. *Proc. VLDB Endow.* 5, 3 (2011), 253–264. <https://doi.org/10.14778/2078331.2078340>
- [42] Quanzhong Li, Minglong Shao, Volker Markl, Kevin S. Beyer, Latha S. Colby, and Guy M. Lohman. 2007. Adaptively Reordering Joins during Query Execution. In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15–20, 2007*, Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 26–35. <https://doi.org/10.1109/ICDE.2007.367848>
- [43] Willi Mann and Nikolaus Augsten. 2014. PEL: Position-Enhanced Length Filter for Set Similarity Joins. In *Proceedings of the 26th GI-Workshop Grundlagen von Datenbanken, Bozen-Bolzano, Italy, October 21st to 24th, 2014 (CEUR Workshop Proceedings)*, Friederike Klan, Günther Specht, and Johann Gamper (Eds.), Vol. 1313. CEUR-WS.org, 89–94. https://ceur-ws.org/Vol-1313/paper_16.pdf
- [44] Willi Mann, Nikolaus Augsten, and Panagiotis Bours. 2016. An Empirical Evaluation of Set Similarity Join Techniques. *Proc. VLDB Endow.* 9, 9 (2016), 636–647. <https://doi.org/10.14778/2947618.2947620>
- [45] Arturas Mazeika, Michael H. Böhlen, Nick Koudas, and Divesh Srivastava. 2007. Estimating the selectivity of approximate string queries. *ACM Trans. Database Syst.* 32, 2 (2007), 12. <https://doi.org/10.1145/1242524.1242529>
- [46] Manoj Muniswamaiah, Tilak Agerwala, and Charles C. Tappert. 2023. Applications of Binary Similarity and Distance Measures. *arXiv:2307.00411 [cs.CV]* <https://arxiv.org/abs/2307.00411>
- [47] Axel-Cyrille Ngonga Ngomo, Mohamed Ahmed Sherif, Kleanthi Georgala, Mofeed Mohamed Hassan, Kevin Dreßler, Klaus Lyko, Daniel Obraczka, and Tommaso Soru. 2021. LIMES: A Framework for Link Discovery on the Semantic Web. *Künstliche Intell.* 35, 3 (2021), 413–423. <https://doi.org/10.1007/S13218-021-00713-X>
- [48] Santiago Ontañón. 2020. An overview of distance and similarity functions for structured data. *Artif. Intell. Rev.* 53, 7 (2020), 5309–5351. <https://doi.org/10.1007/S10462-020-09821-W>
- [49] Nataliya Prokoshyna, Jaroslaw Szlichta, Fei Chiang, Renée J. Miller, and Divesh Srivastava. 2015. Combining Quantitative and Logical Data Cleaning. *Proc. VLDB Endow.* 9, 4 (2015), 300–311. <https://doi.org/10.14778/2856318.2856325>
- [50] Jianbin Qin, Wei Wang, Yifei Lu, Chuan Xiao, and Xuemin Lin. 2011. Efficient exact edit similarity query processing with the asymmetric signature scheme. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2011, Athens, Greece, June 12–16, 2011*, Timos K. Sellis, Renée J. Miller, Anastasios Kementsietsidis, and Yannis Velegrakis (Eds.). ACM, 1033–1044. <https://doi.org/10.1145/1989323.1989431>
- [51] Chuitian Rong, Wei Lu, Xiaoli Wang, Xiaoyong Du, Yueguo Chen, and Anthony K. H. Tung. 2013. Efficient and Scalable Processing of String Similarity Join. *IEEE Trans. Knowl. Data Eng.* 25, 10 (2013), 2217–2230. <https://doi.org/10.1109/TKDE.2012.195>
- [52] Daniel Ulrich Schmitt, Daniel Kocher, Nikolaus Augsten, Willi Mann, and Alexander Miller. 2023. A Two-Level Signature Scheme for Stable Set Similarity Joins. *Proc. VLDB Endow.* 16, 11 (2023), 2686–2698. <https://doi.org/10.14778/3611479.3611480>
- [53] Avi Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts, Seventh Edition*. McGraw-Hill Book Company. <https://www.db-book.com/>
- [54] Yasin N. Silva, Walid G. Aref, Per-Åke Larson, Spencer Pearson, and Mohamed H. Ali. 2013. Similarity queries: their conceptual evaluation, transformations, and processing. *VLDB J.* 22, 3 (2013), 395–420. <https://doi.org/10.1007/s00778-012-0296-4>
- [55] David Silver. 2009. *Reinforcement Learning and Simulation-Based Search in Computer Go*. Ph.D. Dissertation. University of Alberta.
- [56] Ji Sun, Guoliang Li, and Nan Tang. 2021. Learned Cardinality Estimation for Similarity Queries. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20–25, 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 1745–1757. <https://doi.org/10.1145/3448016.3452790>
- [57] Ji Sun, Zeyuan Shang, Guoliang Li, Zhifeng Bao, and Dong Deng. 2019. Balance-Aware Distributed String Similarity-Based Query Processing System. *Proc. VLDB Endow.* 12, 9 (2019), 961–974. <https://doi.org/10.14778/3329772.3329774>
- [58] Ji Sun, Zeyuan Shang, Guoliang Li, Dong Deng, and Zhifeng Bao. 2017. Dima: A Distributed In-Memory Similarity-Based Query Processing System. *Proc. VLDB Endow.* 10, 12 (2017), 1925–1928. <https://doi.org/10.14778/3137765.3137810>
- [59] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. ArnetMiner: extraction and mining of academic social networks. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Las Vegas, Nevada, USA, August 24–27, 2008*, Ying Li, Bing Liu, and Sunita Sarawagi (Eds.). ACM, 990–998. <https://doi.org/10.1145/1401890.1402008>
- [60] Yu Tang, Yilun Cai, and Nikos Mamoulis. 2015. Scaling Similarity Joins over Tree-Structured Data. *Proc. VLDB Endow.* 8, 11 (2015), 1130–1141. <https://doi.org/10.14778/2809974.2809976>
- [61] Immanuel Trummer, Junxiong Wang, Ziyun Wei, Deepak Maram, Samuel Moseley, Saehan Jo, Joseph Antonakakis, and Ankush Rayabhari. 2021. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning. *ACM Trans. Database Syst.* 46, 3 (2021), 9:1–9:45. <https://doi.org/10.1145/3464389>
- [62] Carl A. Waldspurger and William E. Weihl. 1994. Lottery Scheduling: Flexible Proportional-Share Resource Management. In *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI), Monterey, California, USA, November 14–17, 1994*. USENIX Association, 1–11. <https://dl.acm.org/citation.cfm?id=1267639>
- [63] Jiannan Wang, Guoliang Li, and Jianhua Feng. 2012. Can we beat the prefix filtering?: an adaptive framework for similarity join and search. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20–24, 2012*, K. Selçuk Candan, Yi Chen,

- Richard T. Snodgrass, Luis Gravano, and Ariel Fuxman (Eds.). ACM, 85–96. <https://doi.org/10.1145/2213836.2213847>
- [64] Wei Wang, Jianbin Qin, Chuan Xiao, Xuemin Lin, and Heng Tao Shen. 2013. VChunkJoin: An Efficient Algorithm for Edit Similarity Joins. *IEEE Trans. Knowl. Data Eng.* 25, 8 (2013), 1916–1929. <https://doi.org/10.1109/TKDE.2012.79>
- [65] Xubo Wang, Lu Qin, Xuemin Lin, Ying Zhang, and Lijun Chang. 2019. Leveraging set relations in exact and dynamic set similarity join. *VLDB J.* 28, 2 (2019), 267–292. <https://doi.org/10.1007/S00778-018-0529-2>
- [66] Yaoshu Wang, Jianbin Qin, and Wei Wang. 2017. Efficient Approximate Entity Matching Using Jaro-Winkler Distance. In *Web Information Systems Engineering - WISE 2017 - 18th International Conference, Puschino, Russia, October 7-11, 2017, Proceedings, Part I (Lecture Notes in Computer Science)*, Athman Bouguettaya, Yunjun Gao, Andrey Klimenko, Lu Chen, Xiangliang Zhang, Fedor Dzerzhinskiy, Weijia Jia, Stanislav V. Klimenko, and Qing Li (Eds.), Vol. 10569. Springer, 231–239. https://doi.org/10.1007/978-3-319-68783-4_16
- [67] Yaoshu Wang, Chuan Xiao, Jianbin Qin, Xin Cao, Yifang Sun, Wei Wang, and Makoto Onizuka. 2020. Monotonic Cardinality Estimation of Similarity Selection: A Deep Learning Approach. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 1197–1212. <https://doi.org/10.1145/3318464.3380570>
- [68] Manuel Widmoser, Daniel Kocher, Nikolaus Augsten, and Willi Mann. 2023. MetricJoin: Leveraging Metric Properties for Robust Exact Set Similarity Joins. In *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 1045–1058. <https://doi.org/10.1109/ICDE55515.2023.00085>
- [69] Chuan Xiao, Wei Wang, and Xuemin Lin. 2008. Ed-Join: an efficient algorithm for similarity joins with edit distance constraints. *Proc. VLDB Endow.* 1, 1 (2008), 933–944. <https://doi.org/10.14778/1453856.1453957>
- [70] Chuan Xiao, Wei Wang, Xuemin Lin, and Haichuan Shang. 2009. Top-k Set Similarity Joins. In *Proceedings of the 25th International Conference on Data Engineering, ICDE 2009, March 29 2009 - April 2 2009, Shanghai, China*, Yannis E. Ioannidis, Dik Lun Lee, and Raymond T. Ng (Eds.). IEEE Computer Society, 916–927. <https://doi.org/10.1109/ICDE.2009.111>
- [71] Chuan Xiao, Wei Wang, Xuemin Lin, Jeffrey Xu Yu, and Guoren Wang. 2011. Efficient similarity joins for near-duplicate detection. *ACM Trans. Database Syst.* 36, 3 (2011), 15:1–15:41. <https://doi.org/10.1145/2000824.2000825>
- [72] Rui Yang, Panos Kalnis, and Anthony K. H. Tung. 2005. Similarity Evaluation on Tree-structured Data. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, Fatma Özcan (Ed.). ACM, 754–765. <https://doi.org/10.1145/1066157.1066243>
- [73] Haoyu Zhang and Qin Zhang. 2017. EmbedJoin: Efficient Edit Similarity Joins via Embeddings. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*. ACM, 585–594. <https://doi.org/10.1145/3097983.3098003>
- [74] Haoyu Zhang and Qin Zhang. 2019. MinJoin: Efficient Edit Similarity Joins via Local Hash Minima. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, Ankur Teredesai, Vipin Kumar, Ying Li, Rómer Rosales, Evimaria Terzi, and George Karypis (Eds.). ACM, 1093–1103. <https://doi.org/10.1145/3292500.3330853>